

SMM Plan

Комплексный аудит проекта

Безопасность / Логика / Маркетинг /
Архитектура

Дата: 2026-04-10

Репозиторий: github.com/a9040000911-lgtm/SMM_plan

С о д е р ж а н и е

1. Резюме проекта	3
2. Рекомендуемый перечень узкоспециализированных аудитов	4
3. Аудит безопасности	7
3.1. Аутентификация и авторизация	7
3.2. Финансовая безопасность	9
3.3. API безопасность	10
3.4. Защита данных	11
4. Логические ошибки	12
4.1. Финансовая логика	12
4.2. Бизнес-логика заказов	13
4.3. Мульти-тенантная изоляция	14
5. Маркетинговый аудит	14
5.1. Реферальная программа	14
5.2. Промо-система и лояльность	15
5.3. Upsell и cross-sell механики	16
5.4. Геймификация и удержание	17
6. Архитектурные наблюдения	18
6.1. Позитивные аспекты	18
6.2. Проблемные области	19
7. Приоритетная дорожная карта исправлений	19

1. Резюме проекта

SMM Plan (Smmplan) представляет собой высококонверсионную B2B/B2C SaaS-платформу для управления SMM-услугами, автоматизации заказов и управления лояльностью клиентов. Проект построен на стеке Next.js 16 + React 19 + TypeScript + Prisma ORM + PostgreSQL + Redis. Платформа объединяет веб-приложение, Telegram-бота, административную панель и интеграции с внешними SMM-провайдерами. Исходный код содержит порядка 2700+ файлов, что свидетельствует о значительной зрелости и сложности проекта. Бизнес-модель ориентирована на реселлеров и конечных потребителей SMM-услуг (подписчики, лайки, просмотры и пр.) с поддержкой мульти-тенантной архитектуры.

В ходе частичного аудита были выявлены уязвимости различной степени критичности: от критических проблем аутентификации и авторизации до логических ошибок в финансовых операциях и маркетинговых механизмах. Ниже представлена сводная таблица выявленных проблем, сгруппированных по категориям. Общее количество найденных проблем составляет 32, из которых 6 относятся к категории критических, 11 – к высоким, 10 – к средним и 5 – к низким.

Категория	Критически	Высокие	Средние	Низкие
Аутентификация и авторизация	3	2	1	0
Финансовая безопасность	1	3	2	0
API и интеграции	1	2	2	1

Логические ошибки	1	2	2	2
Маркетинг и лояльность	0	2	3	2

Таблица 1. Сводная таблица выявленных проблем по категориям

2. Рекомендуемый перечень узкоспециализированных аудитов

Для проведения полноценного всестороннего аудита проекта SMM Plan рекомендуется провести следующие узкоспециализированные аудиты. Каждый аудит сфокусирован на конкретной области и позволяет выявить проблемы, которые могут быть упущены при общем обзорном подходе. Данный перечень составлен на основе анализа архитектуры проекта, его бизнес-логики и технологического стека.

A01. Аудит аутентификации и управления сессиями

Проверка механизмов входа, 2FA, JWT-токенов, magic-ссылок, управления сессиями, cookie-политик, предотвращения session fixation и session hijacking. Анализ админской и пользовательской аутентификации, мульти-тенантной изоляции сессий.

A02. Аудит авторизации и контроля доступа

Проверка RBAC-модели, проверка прав доступа на каждом API-endpoint, анализ горизонтальной и вертикальной эскалации привилегий, проверка project-scoping, API-key авторизации.

A03. Аудит веб-хуков и платёжной безопасности

Проверка целостности webhook-уведомлений от YooKassa/Robokassa, защита от повторной обработки (idempotency), верификация сумм, защита от манипуляций с metadata, анализ race conditions при параллельных пополнениях.

A04. Аудит API безопасности (V1/V2)

Проверка публичных API на rate limiting, валидацию входных данных, информационные утечки в error messages, защиту от mass assignment, контроль доступа по API-key.

A05. Аудит финансовой логики и бухгалтерии

Проверка атомарности транзакций (списание/начисление), двойной бухгалтерии (Ledger), расчёта цен, округлений Decimal, race conditions при балансовых операциях, slippage-анализ провайдеров.

A06. Аудит реферальной и промо-системы

Проверка защиты от фрода в реферальной программе (self-referral, мульти-аккаунтинг), валидация промокодов, ограничение применения, защита от brute-force промокодов, идемпотентность начисления бонусов.

A07. Аудит Telegram-бота (Telegraf)

Проверка валидации Telegram initData, защиты от подделки callback-query, анализ wizard-сцен на предмет пропуска шагов валидации, проверка rate limiting в bot-командах.

A08. Аудит мульти-тенантной изоляции

Проверка project-scoping на всех уровнях:
БД-запросы, API, UI, сессии. Анализ возможных data

leaks между тенантами, проверка tenant-resolution логики.

A09. Аудит обработки заказов и провайдеров

Проверка маршрутизации заказов к провайдерам, fallback-логики, автоматического мониторинга (AutoMonitoring), drip-feed, обработки статусов, retry-механизмов.

A10. Аудит администраторской панели

Проверка CSRF-защиты, XSS в admin-действиях, защиты от mass-действий, аудит-логирование действий, валидация входных данных в server actions.

A11. Аудит инфраструктуры и деплоя

Проверка Docker-конфигураций, nginx-настроек, secret management, CI/CD pipeline, backup-стратегии, мониторинга и алертинга.

A12. Аудит GDPR/законодательной compliance

Проверка обработки персональных данных, прав на удаление, cookie-политики, пользовательских соглашений, логирования доступа к данным.

A13. Аудит маркетинговой воронки и конверсии

Анализ user journey от первого визита до повторной покупки, проверка upsell/cross-sell механик, checkout flow, abandoned cart, email-триггеров.

A14. Аудит SEO и контент-стратегии

Проверка SSR/SSG, meta-тегов, sitemap, robots.txt, микроразметки, скорости загрузки, мобильной адаптивности, CMS-возможностей.

A15. Аудит производительности и масштабировемости

Анализ N+1-запросов, индексов БД, кэширования (Redis), rate limiting, пагинации, загрузки ассетов, WebSocket-нагрузки.

3. Аудит безопасности

3.1. Аутентификация и авторизация

[CRITICAL] C-01: Захардкоженный мастер-ключ и email в 2FA

В файле `src/app/api/admin/auth/route.ts` (строка 55) обнаружена критическая уязвимость: мастер-ключ для обхода двухфакторной аутентификации привязан к конкретному email-адресу `art@artmspektr.ru`. Это означает, что любой, кто знает `ADMIN_MASTER_KEY`, может полностью обойти 2FA для данного аккаунта. Кроме того, email-адрес захардкожен в бизнес-логике, что нарушает принцип разделения конфигурации и кода. Если мастер-ключ будет скомпрометирован (утечка через `.env`, логи, дампы памяти), злоумышленник получит полный доступ к административной панели без возможности обнаружения через стандартные механизмы 2FA, поскольку логирование помечает такой вход как легитимный.

[CRITICAL] C-02: Отключённый 2FA для админских аккаунтов

В том же файле (строки 233–247) обнаружена логика обхода двухфакторной аутентификации: если у пользователя поле `twoFactorEnabled = false`, то 2FA полностью пропускается, и сессия создаётся немедленно после проверки пароля. Это создает вектор атаки: если злоумышленник сможет изменить значение этого поля в базе данных (через SQL-инъекцию, компрометацию БД или уязвимость в admin API), он получит доступ без

второго фактора. Комментарий в коде гласит "This is intentional for test accounts and trusted devices", однако в продакшн-среде такой механизм неприемлем. Рекомендуется enforce 2FA для всех административных аккаунтов без исключений, а для тестовых сред использовать отдельный механизм.

[CRITICAL] C-03: Отсутствие Next.js middleware

В проекте отсутствует стандартный файл `middleware.ts`. Вместо этого используется `src/proxy.ts` (Next.js 16 convention), который выполняет функции авторизации, rate limiting и мульти-тенантного роутинга. Однако проверка авторизации в proxy-логике проверяет только наличие валидного JWT в cookie, не проверяя при этом роль пользователя для каждого конкретного маршрута. Это означает, что любой аутентифицированный пользователь с валидной сессией может потенциально получить доступ к API-маршрутам, предназначенным только для администраторов, если серверный обработчик не выполняет дополнительную проверку ролей.

[HIGH] H-01: Информационные утечки в логах аутентификации

В файле `src/services/core/auth.ts` (строки 76-77) обнаружены debug-логи, которые выводят email пользователя и информацию о наличии пароля в консоль сервера. В продакшн-среде это может привести к утечке персональных данных через системы агрегации логов. Кроме того, в `src/app/api/admin/auth/route.ts` множественные `console.log` с отладочной информацией о времени выполнения каждого шага аутентификации.

[HIGH] H-02: Слабая генерация паролей для новых пользователей

В файле `src/app/api/client/orders/route.ts` (строки 184, 468) для новых пользователей генерируется пароль через `Math.random().toString(36).slice(-8)`. Функция `Math.random()` не является криптографически безопасной и может быть предсказуема. Длина пароля составляет лишь 16 символов (два конкатенированных блока по 8), но энтропия значительно ниже из-за особенностей `base36`-кодирования. Рекомендуется использовать `crypto.randomBytes()` или аналогичные криптографически стойкие генераторы.

3.2. Финансовая безопасность

[CRITICAL] C-04: Race condition при параллельных заказах без балансовой блокировки

В файле `src/app/api/client/orders/route.ts` для одиночного заказа (строки 547–603) баланс проверяется вне транзакции через `prisma.user.findUnique`, а затем списание выполняется в отдельной транзакции. Между проверкой и списанием может пройти время, в течение которого параллельный запрос может дважды использовать один и тот же баланс. Хотя внутри транзакции используется `updateMany` с условием `balance: { gte: totalPrice }`, это защищает только от отрицательного баланса, но не от двойного расходования при конкурентных запросах, если оба запроса успеют прочитать одинаковый баланс до обновления. В `batch`-обработке (строки 252–312) эта проблема решена лучше через `tx.user.findUnique` внутри транзакции, однако и там остаётся окно для `TOCTOU`-атаки между чтением и обновлением.

[HIGH] H-03: Небезопасная передача платёжных реквизитов в клиентском коде

В файле `src/app/api/client/orders/route.ts` (строки 371–376, 629–634) платёжные креденшалы (`shopId`, `secretKey`) извлекаются из настроек проекта в `paymentSettings` и передаются непосредственно в `PaymentService`. Хотя эта логика выполняется на сервере, сам факт хранения секретных ключей YooKassa в JSON-полле `project.paymentSettings` базы данных создаёт риск утечки при компрометации БД. Рекомендуется использовать шифрование на уровне приложения (в проекте есть `CryptoService.decrypt`, но неясно, всегда ли он применяется при сохранении).

[HIGH] H-04: Duplicate variable declaration в YooKassa webhook

В файле `src/app/api/webhooks/yookassa/route.ts` (строка 54) обнаружено дублирование объявления переменной `isSimulation` – она объявлена дважды: на строке 41 и на строке 54. JavaScript позволяет повторное объявление через `var`, но при использовании `let/const` это вызовет синтаксическую ошибку. В данном случае используется тот же паттерн, что может привести к непредсказуемому поведению в разных runtime-средах.

3.3. API безопасность

[CRITICAL] C-05: Слабая авторизация в Internal API

В файле `src/app/api/internal/global-settings/route.ts` обнаружена слабая авторизация: если `INTERNAL_API_KEY` не задан, доступ к API разрешён с любого `localhost`-адреса. Условие `if (internalKey && authHeader !== ...)` означает, что при отсутствии ключа проверка полностью пропускается. Кроме того, проверка IP-адреса на `127.0.0.1` уязвима к IP-спуфингу через заголовок `X-Forwarded-For` в конфигурациях с обратным прокси. Этот эндпоинт даёт доступ ко всем глобальным

настройкам системы, включая лимиты, ключи и конфигурацию.

[HIGH] H-05: Отсутствие валидации ownership в V1 API

В файле `src/app/api/v1/orders/route.ts` `projectId` устанавливается в `null` (строка 78), что означает отсутствие проектного контекста при обработке заказов через публичный API. Это может привести к созданию заказов вне проектного контекста, нарушению мульти-тенантной изоляции и некорректному расчёту цен. Кроме того, API-ключ используется как единственный фактор аутентификации, что при утечке ключа даёт полный доступ к аккаунту без возможности дополнительной верификации.

[MEDIUM] M-01: Debug-эндпоинт доступен в development-режиме

Файл `src/app/api/debug-session/route.ts` предоставляет информацию о сессии администратора, но ограничен только режимом `development` через проверку `NODE_ENV`. Риск заключается в том, что при ошибочной конфигурации продакшн-сервера (`NODE_ENV=development`) этот эндпоинт станет доступен извне. Рекомендуется дополнительно ограничить доступ по IP-адресу или удалить debug-эндпоинты из продакшн-сборки.

3.4. Защита данных

[HIGH] H-06: Хранение 2FA-кодов в открытом виде в БД

В файле `src/app/api/admin/auth/route.ts` (строки 263-272) 2FA-коды сохраняются в базе данных в открытом виде (поля `twoFactorCode` и `twoFactorExpires` модели `User`). При компрометации базы данных злоумышленник сможет использовать сохранённые коды для входа в административную панель в течение 5 минут (время жизни кода). Рекомендуется хэшировать коды перед сохранением,

аналогично хранению паролей, и проверять через функцию сравнения хэшей.

[MEDIUM] M-02: GPG-ключи в репозитории

В директории `scripts/.gnupg/` обнаружены GPG-ключи (включая приватные ключи в `private-keys-v1.d`). Хотя эти файлы могли быть добавлены случайно, их наличие в публичном репозитории является серьёзной утечкой криптографических материалов. Рекомендуется немедленно удалить эти ключи из истории коммитов с помощью `git filter-branch` или `BFG Repo-Cleaner`, а также отозвать скомпрометированные ключи.

4. Логические ошибки

4.1. Финансовая логика

[CRITICAL] C-06: Несоответствие в расчёте баланса в `ledger-записях`

В файле `src/app/api/client/orders/route.ts` при одиночном заказе (строки 548–575) значения `balanceBefore` и `balanceAfter` вычисляются ДО выполнения транзакции (на основе данных из `findUnique`), однако фактическое списание происходит внутри транзакции через `updateMany`. Если между `findUnique` и транзакцией произойдёт параллельное изменение баланса (другой заказ, пополнение, бонус), записанные в `LedgerEntry` значения `balanceBefore` и `balanceAfter` будут неактуальными. Это создаёт рассинхронизацию в бухгалтерской книге и может привести к невозможности корректного аудита финансовых операций. Сервис `FinancialSecurityService.analyzeUserLTV` пытается обнаружить такие рассинхронизации, но ретроактивно.

[HIGH] H-07: Отсутствие проверки `scheduled order` баланса

При создании запланированного заказа (строки 522–543 в том же файле) не проверяется

достаточность средств на балансе пользователя и не создаётся блокировка средств. Заказ создаётся со статусом `PENDING`, но без какой-либо гарантии, что на момент исполнения у пользователя будет достаточно средств. Это позволяет создавать неограниченное количество запланированных заказов, которые при исполнении могут привести к отрицательному балансу или массовым отказам.

4.2. Бизнес-логика заказов

[HIGH] Н-08: Заказ без аутентификации создаёт пользователя с случайным паролем

Механизм гостевого заказа (строки 160–192, 444–481) позволяет создать заказ без предварительной регистрации. При этом для нового `email` автоматически создаётся пользователь с случайно сгенерированным паролем, который отправляется на указанный `email`. Это создаёт несколько проблем: во-первых, пользователь может не заметить письмо с паролем, и его аккаунт останется незащищённым; во-вторых, при компрометации почтового ящика злоумышленник получит доступ к аккаунту; в-третьих, `magic-токен`, возвращаемый в ответе на заказ, действителен 5 минут и позволяет автоматически войти в систему без пароля.

[MEDIUM] М-03: Race condition в промо-валидации

В файле `src/services/users/promo.service.ts` метод `validatePromo` проверяет активность промокода и его использование, но не использует транзакцию или блокировку. При конкурентных запросах один и тот же промокод может быть применён дважды до того, как будет отмечен как использованный. Аналогичная проблема

существует в `processAutomationRules`, где идемпотентность проверяется через `SystemLog`, но без транзакционной гарантии.

4.3. Мульти-тенантная изоляция

[MEDIUM] M-04: Tenant resolution через cookie может быть подделан

В файле `src/utils/admin-session.ts` (строки 74-95) активный проект определяется через cookie `active_project_id`. Хотя проверяется, что проект входит в список разрешённых (`allowedProjects`), для глобальных администраторов (`isGlobalAdmin`) такая проверка не выполняется, и они могут переключиться на любой проект, включая чужие. Кроме того, значение "all" позволяет получить доступ ко всем проектам без дополнительной верификации.

5. Маркетинговый аудит

5.1. Реферальная программа

Реферальная система реализована в `src/services/users/referral.service.ts` и включает tiered-бонусы, ежемесячный лидерборд и систему достижений. Анализ выявил следующие проблемы с точки зрения маркетинговой эффективности и защиты от фрода.

[HIGH] H-09: Отсутствие защиты от self-referral

Система не проверяет, что реферер и реферал являются разными пользователями. Пользователь может создать множество аккаунтов с разными email-адресами и использовать свой реферальный код для саморегистрации, получая бонусы с каждого пополнения "приглашённого" аккаунта. В модели `User` поле `referralCode` генерируется

автоматически как `UUID`, но нет механизма связывания аккаунтов по `IP`, устройству или платёжным данным для обнаружения мульти-аккаунтинга. Также отсутствует лимит на количество рефералов с одного `IP`-адреса.

[MEDIUM] M-05: Некорректный расчёт реферального кода

В методе `getReferralDashboardData` (строка 129) реферальный код пользователя вычисляется как `user.id.split("-")[0]`, то есть берётся только первая часть `UUID`. Это не соответствует реальному `referralCode`, хранящемуся в базе данных (полный `UUID`). Пользователи будут делиться неверным реферальным кодом, что приведёт к потере конверсии и некорректной атрибуции.

5.2. Промо-система и лояльность

Промо-система реализована в `src/services/users/promo.service.ts` и включает автоматические триггеры (новый клиент, крупное пополнение, траты), а также динамические `automation rules`.

[HIGH] H-10: Промокоды без ограничений использования

В модели `PromoCode` отсутствуют поля `maxUsageCount` и `currentUsageCount`. Это означает, что любой активный промокод может быть использован неограниченное количество раз разными пользователями. Кроме того, в методе `validatePromo` проверяется только, что конкретный пользователь не использовал код ранее (через `UserPromo`), но не проверяется общее количество применений. Для масштабной маркетинговой кампании это может привести к значительным финансовым потерям, если промокод станет

публичным.

[MEDIUM] M-06: Предсказуемая генерация промокодов

Промокоды генерируются по шаблону `PROMO{percent}-{random}`, где `random` состоит из 7 символов, сгенерированных через `Math.random().toString(36)`. Энтропия такого кода составляет примерно $36^7 = 78$ млрд комбинаций, но с учётом предсказуемости `Math.random()` и фиксированного префикса `PROMO5-` или `PROMO10-`, атака `brute-force` становится более реалистичной. Отсутствует `rate limiting` на эндпоинт валидации промокодов.

[MEDIUM] M-07: Незавершённая статистика реферальной дашборда

В `getReferralDashboardData` (строки 138–143) поля `currentMonthCount`, `currentMonthRevenue` и `currentRank` возвращаются за хардкоженными как 0, "0" и `null` соответственно. `Tier breakdown` также возвращает некорректные данные (`tier2` и `tier3` всегда 0). Это делает дашборд реферальной программы практически бесполезным для пользователей и снижает мотивацию к участию в программе.

5.3. Upsell и cross-sell механики

[MEDIUM] M-08: Upsell API без аутентификации

Эндпоинт `/api/client/marketing/upsell` (файл `src/app/api/client/marketing/upsell/route.ts`) не требует аутентификации. Хотя данные не являются критичными, отсутствие авторизации позволяет конкурентам автоматически сканировать каталог услуг и структуру кросс-селл предложений. Рекомендуется добавить хотя бы базовую аутентификацию или `rate limiting` на данный эндпоинт.

[LOW] L-01: Примитивный алгоритм cross-sell

Текущий алгоритм upsell предлагает самую дешёвую услугу из другой категории на той же платформе. Это маркетингово неэффективно: самый дешёвый вариант не обязательно имеет наибольшую ценность для клиента. Рекомендуется внедрить персонализированные рекомендации на основе истории заказов, популярности услуг и среднего чека.

5.4. Геймификация и удержание

Система геймификации включает достижения (Achievement), челленджи (Challenge), NPS-опросы и систему отзывов с вознаграждениями. Анализ показал следующие маркетинговые проблемы.

[LOW] L-02: Отсутствие drip-кампаний и email-триггеров

В проекте отсутствует система автоматических email-рассылок для реактивации пользователей, abandoned cart-напоминаний и onboarding-последовательностей. При наличии nodemailer в зависимостях и инфраструктуры для отправки писем, маркетинговые триггеры не реализованы. Это упускает значительный потенциал конверсии: по отраслевым данным, abandoned cart-напоминания повышают конверсию на 10–15%.

[LOW] L-03: NPS-опросы без автоматической маршрутизации

Система NPS-опросов (модель NPSSurvey) собирает данные, но не имеет автоматической маршрутизации: детракторы (оценка 0–6) не получают автоматических предложений компенсации, пассивные пользователи (7–8) не получают стимулов для повышения лояльности, а промоутеры (9–10) не получают реферальных приглашений. Ручная обработка NPS-данных снижает эффективность программы лояльности.

[LOW] L-04: Bug Bounty программа с потенциальным фродом

Модель BugReport позволяет пользователям сообщать о багах и получать вознаграждение. Однако отсутствует защита от фрода: пользователь может сообщить о несуществующем баге, получить награду (rewardAmount), и поскольку поле reviewedBy не проверяется автоматически, модерация может быть недостаточной.

6. Архитектурные наблюдения

6.1. Позитивные аспекты

Проект демонстрирует ряд качественных архитектурных решений, которые заслуживают внимания и могут служить примером для других проектов аналогичного масштаба. Во-первых, реализована двойная бухгалтерия через LedgerService и LedgerEntry, что обеспечивает возможность аудита всех финансовых операций. Во-вторых, внедрена система FinancialSecurityService с мониторингом slippage провайдеров и LTV-анализом пользователей, что позволяет обнаруживать финансовые аномалии. В-третьих, webhook YooKassa реализован с несколькими уровнями защиты: IP-валидация, повторная верификация статуса через API, идемпотентность через БД-поиск дублей, и атомарные транзакции с row-level блокировками. В-четвёртых, используется Zod-валидация входных данных API (clientCheckoutSchema, v2AddOrderSchema), что снижает риск инъекций и malformed-запросов. В-пятых, реализована rate limiting через Upstash Redis с fallback на in-memory для разработки.

6.2. Проблемные области

Среди архитектурных проблем следует отметить: отсутствие единой `middleware` для авторизации (проверки разбросаны по разным маршрутам), смешивание административной и клиентской аутентификации в `admin-session.ts` (fallback с `admin_session` на `NextAuth`), отсутствие сервисного слоя для заказов (значительная логика размещена непосредственно в `route.ts`), дублирование кода между `batch-` и `single-order` обработкой, отсутствие чёткой доменной модели для финансовых операций.

7. Приоритетная дорожная карта исправлений

На основе выявленных проблем формируется следующая приоритетная дорожная карта исправлений, ранжированная по степени критичности и потенциальному бизнес-влиянию.

П р и о р и т е т	Проблема	Действие	С р о к
P0	C-01: Мастер-ключ + email	Убрать захардкоженный email, добавить ротацию ключей	1 день
P0	C-04: Race condition баланса	Row-level lock + SELECT FOR UPDATE	2 дня
P0	C-05: Internal API авторизация	Обязательный INTERNAL_API_KEY, убрать IP-fallback	1 день
P0	C-06: Ledger рассинхронизация	Вычислять balanceBefore/After внутри tx	2 дня

P1	C-02: Обход 2FA	Enforce 2FA для всех ADMIN	1 день
P1	C-03: Middleware авторизация	Ролевая проверка в proxy/middleware	3 дня
P1	H-06: 2FA коды в открытом виде	Хэшировать коды через bcrypt	1 день
P1	H-09: Self-referral фрод	Антифрод по IP/device fingerprint	3 дня
P1	M-02: GPG ключи в репо	BFG Repo-Cleaner + отзыв ключей	1 день
P2	H-07: Scheduled order баланс	Предавторизация суммы / escrow	3 дня
P2	H-10: Лимиты промокодов	Добавить maxUsageCount + текущий счётчик	2 дня
P2	M-05: Реферальный код	Использовать реальный referralCode из БД	0.5 дня
P2	H-01: Debug логи	Убрать console.log, использовать winston/pino	2 дня
P3	M-06: Промокод энтропия	crypto.randomBytes + rate limit	1 день
P3	L-02: Drip-кампании	Интегрировать email-automation	2 недели

Таблица 2. Приоритетная дорожная карта исправлений